

Emergency Dispatch Priority and Coordination System

Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Problem Definition and Approval.....	3
1.1 Background, Context and Stakeholders.....	3
1.2 Problem Statement and Software Quality Issues.....	4
1.3 Why the problem is suitable for a Software Quality Assurance Project.....	5
2. Proposed Solution.....	6
2.1 Proposed solution.....	6
2.2 How the proposed solution addresses the selected real world problem.....	7
3. Requirements and Quality Analysis.....	8
3.1 Functional Requirements.....	8
3.2 Non-Functional Requirements.....	9
3.3 Requirements Analysis.....	10
3.4 Improved and more testable requirements.....	15
3.5 Defined acceptance criteria for the key features.....	18
3.6 Identify relevant software quality attributes (maybe delete this section).....	20
4. Initial Prototype.....	21
5. AI-Assisted Development Using GitHub Copilot.....	22
6. Initial Test Strategy and Test Planning.....	22
6.1 Scope of Testing.....	22
6.2 Test Types / Levels.....	23
6.3 Functional testing approach.....	25
6.4 Non-Functional Testing Approach.....	25
6.5 Entry and Exit Criteria.....	26
6.6 Test environment, tools, and dependencies.....	26
7. Initial Test Cases and Requirements Traceability.....	27
8. Project Progress, Risks, and Next Steps.....	28
Appendix.....	29

1. Problem Definition and Approval

Note:

- Need a specific client for what the system targets
- Need to be for a certain location e.g Auckland?
- He requested that our scope is too small (include authorization, caller login to submit their own record for the dispatcher,
- Briefly state the quality attribute in the problem statement, and then further explain it in Task 2.
-

1.1 Background, Context and Stakeholders

The emergency dispatch systems answer emergency calls, record details about the emergencies, and dispatch the right units accordingly. This project is inspired by real-world emergency dispatch; however, it is a simplified hypothetical model that does not represent an actual system. This will be used throughout this project.

The main stakeholders in the modelled emergency dispatch system are the **Caller, Dispatcher, Field Responders**, and **Software Developers**. The dispatch process begins with the Caller, who contacts the emergency service and provides information about the situation. The Dispatcher receives this information, records the emergency as a case, manually assigns its severity or priority, and determines which response units are required. The dispatcher then directly assigns available Field Responders, including ambulance, police, and fire units, to the case. Software Developers are also stakeholders as they are responsible for maintaining, testing, and improving the dispatch system.

The emergency dispatch software has significant practical relevance. Most importantly, a dispatch system's quality can have an impact on the effectiveness of an emergency. Failing to respond to a case punctually, assigning the wrong priority/severity, or poor coordination of response units could lead to slower emergency responses. Because emergency dispatching is a life-critical process, the software that supports it must operate accurately and consistently under pressure.

1.2 Problem Statement and Software Quality Issues

Managing emergency dispatching requires all aspects of the system to be of high quality because, unlike other systems, the emergency dispatch system is life-critical and requires immediate action. Any compromises in quality, such as issues in **reliability, usability, or maintainability**, could potentially come with the loss of lives.

The modelled emergency dispatch system relies on manual priority allocation by the dispatcher for incoming cases. This manual assignment could cause inconsistencies in priorities, resulting in low-priority cases being handled first and potential delays, **compromising the reliability** of the system.

For systems at the scale of emergency dispatching, it becomes difficult to control, especially since there are thousands of calls daily. Within the modelled system, dispatchers do not have an efficient way to record and manage call logs. The dispatcher also has to directly communicate and manually dispatch units to an ongoing case. At the scale of an emergency dispatching system, the number of requests for units becomes overwhelming and hard to manage, **compromising usability**.

The modelled system does not automatically log calls, meaning dispatchers need to do it manually, which takes time during the call. The current system for callers to dispatchers sometimes goes down for a few seconds at a time during the day, which **undermines reliability** since some calls may not come through.

The current system consists of multiple dispatchers all working towards assigning cases to dispatched units. However, this is a **reliability issue** as it opens up potential race condition vulnerabilities where a dispatched unit may be assigned to multiple cases at once.

Every emergency dispatch system needs to be **robust and maintainable**, due to the constant need for system updates. The proposed project's current design has three dispatchable units: ambulance, police, and fire. If we were to add a new unit to our system, such as search and rescue, the current system is not modular and would not be able to accommodate it, requiring large refactoring. Maintainability is an important software quality issue to consider in the design phase since neglecting it increases the complexity of adding new unit types into the system. This, in turn, increases the likelihood of defects being introduced when the system is modified.

1.3 Why the problem is suitable for a Software Quality Assurance Project

Our selected problem is suitable for a Software Quality Assurance project because it contains clear software quality issues that can be analysed and tested. The main quality issues we mentioned in Section 1.2 are reliability, usability, and maintainability, which are also recognised quality characteristics within **ISO/IEC 25010**.

The **reliability issues** include possible race conditions during unit assignment, inconsistent priority allocation, and downtime within the modelled system. The **usability issues** come from the amount of manual work required from dispatchers, including recording calls and coordinating units under high workloads. The **maintainability issue** comes from the structure of the system being non-modular, where adding new units would require large changes to the existing code.

These issues are suitable for quality assurance because they can easily be used to form clear requirements, acceptance criteria, and test cases. This makes it possible to test the system to see if the quality issues have been improved or removed.

Quality assurance is incredibly important for an emergency dispatch system as defects can have significant consequences such as delayed responses, misprioritisation of cases, or dispatching of units to the wrong case. Thus, before we can use a system, it must be reliable, usable, and maintainable.

2. Proposed Solution

2.1 Proposed solution

The proposed project will develop a prototype Emergency Dispatch Priority and Coordination System. The system will include an interface for the dispatcher to record the details of each emergency call and create a case. Instead of the dispatcher manually handling every part of the dispatch process, the proposed system will standardise priority allocation and route each case to the relevant emergency response department or departments.

The proposed solution introduces the emergency departments as an **additional stakeholder within the system**. These departments include areas such as Medical, Police, and Fire. Once the case gets routed, the relevant department will be responsible for managing the case and coordinating its own response units. The system will also keep track of the status and availability of cases and units, allowing each of the respective departments to be informed at all times, so that they can manage them more clearly through a central interface. Response units assigned to the case sign off when they have completed their response. Once all assigned units have signed off, the system closes the case.

The system will also keep a history of previous cases, providing a complete audit trail of each emergency, so that important case information can be searched when needed. The overall design will be more modular so that new departments, response types, or user roles can be added more easily in the future.

2.2 How the proposed solution addresses the selected real world problem.

The proposed solution addresses the reliability issues by reducing the amount of manual decision-making required from the dispatcher. A standardised priority allocator will be used so that the priority of each case is assigned consistently rather than depending on the judgement of individual dispatchers. Routing cases to the relevant departments also creates clearer responsibility for case and unit management, helping reduce the risk of multiple dispatchers attempting to assign the same unit at the same time.

The proposed solution also improves usability by reducing the number of tasks that the dispatcher has to manage manually. The dispatcher will mainly be responsible for recording emergency information and creating the case, while priority allocation, routing, and ongoing case management are handled by the system and the relevant departments. This reduces the workload on the dispatcher and makes the dispatch process easier to manage.

Finally, the system addresses maintainability by using a more modular design. Responsibilities such as call handling, priority allocation, department management, and unit coordination are separated so that changes can be made more easily. This will make it easier to add new departments, response types, or user roles without requiring large changes throughout the existing system. Design principles such as SOLID and suitable design patterns can be used during development to support this modular structure.

3. Requirements and Quality Analysis

3.1 Functional Requirements

- **FR-01** The system shall provide an interface for the dispatcher to record call details, including caller information, incident notice, description of incident, and timestamp.
- **FR-02** The system shall allow the dispatcher to assign a case from a call.
- **FR-03** The system shall manage all dispatched units with each unit's current availability status (not available/available), unit type, location, identifier, and unit personnel count.
- **FR-04** The system shall automatically assign a priority level to each incoming emergency case based on its severity.
- **FR-05** The system shall automatically route a case to the appropriate emergency response department.
- **FR-06** The system shall automatically assign the most suitable unit within the relevant departments for a case.
- **FR-07** The system shall display all the cases on a single live dashboard.
- **FR-08** The system shall update the status of each case throughout the lifecycle of the case.
- **FR-09** The system shall automatically update a unit's availability status when it is assigned or the case is released.
- **FR-10** The system shall notify the assigned case with its details upon assignment.
- **FR-11** The system shall allow the dispatcher to mark a case (resolved/closed), which releases the unit assigned to it.
- **FR-12** The system shall allow the dispatcher to search through the history of logs by typing date, caller, and ID.

3.2 Non-Functional Requirements

Performance (P)
1. NFR-P01: When the dispatcher has assigned a case to the system, the system shall send it to the most suitable department for that case in under 2 seconds. 2. NFR-P02: The system shall allow at least 100 concurrent users without performance issues
Maintainability (M)
1. NFR-M01: The system shall be built using a modular architecture, so if one module fails, it will not cause the entire system to crash. 2. NFR-M02: The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.
Reliability (R)
1. NFR-R01: Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department. 2. NFR-R02: The system shall run at least 99.9% of the time, given that it operates in a life-critical environment 24 hours a day. 3. NFR-R03: All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered. 4. NFR-R04: When there is a large number of cases open simultaneously, the system shall continue to function without crashing.
Security (S)
1. NFR-S01: Any data that passes between users and the system must be encrypted to prevent unauthorised access. 2. NFR-S02: The system shall restrict access to system features based on each user's assigned role, as defined in section 1.1.

Usability (U)
1. NFR-U01: The system shall display a tailored view of the system for each user role, displaying only the tools and information required for that role. 2. NFR-U02: The system shall return all error messages in plain English so they can be easily understood. 3. NFR-U03: All users shall be able to operate the core functions of their role within the system after no more than 1 hour of training.

3.3 Requirements Analysis

The functional and non-functional requirements were analysed using the requirements quality criteria for **clarity, completeness, consistency, correctness, feasibility, and testability**. These criteria are used to identify whether each requirement is clearly defined, consistent with the rest of the system, representative of the intended behaviour, and can be objectively tested. *The following table identifies the main issues found within the current requirements:*

Requirement	Quality Criteria	Analysis
FR-01	Clarity	The problem is the lack of clarity for the term “incident notice”. The type of incident could be clearer; for example, is it a type, address, or notification?
FR-02	Clarity, Correctness	“Assign a case from a call” does not clearly describe the required action. The intended flow is the dispatcher ‘creating’ a case from a call, so ‘assign’ doesn’t accurately reflect that behaviour.
FR-03	Clarity, Completeness, Testability	The word manage is a bit ambiguous in terms of what the system does with the unit’s information. Manage could mean just allowing the dispatcher to view, or does it also update the unit’s details? This makes the required behaviour incomplete and prevents testers from determining objectively what functionality must pass.

FR-04	Completeness, Testability	The requirement states that severity determines priority, but the severity levels, priority levels and rules connecting them are not defined. The conditions and expected priority are therefore incomplete, and a tester cannot objectively determine whether the assigned priority is correct.
FR-05	Clarity, Completeness, Correctness, Testability	“Appropriate emergency response department” does not define how it is routed. The intended structure also allowed for a single case to be routed to multiple departments, but this functional requirement only refers to a single department. This wording does not represent the intended modelled behavior, and the incomplete routing conditions cannot be objectively tested.
FR-06	Clarity, Completeness, Testability	The phrase “most suitable unit” is ambiguous because it does not identify under what circumstances that a unit is the most suitable; it could be availability, proximity, experience, personnel count. Therefore, the expected unit cannot be objectively determined.
FR-07	Clarity, Completeness, Testability	The issue in this requirement is what’s displayed on the dashboard and what “live” means. It is clear that the system needs a dashboard to display all cases, but what exactly does each case display on the dashboard? Also, what live means could be a constant update or just updated when the page is refreshed. The required scope and behaviour of the dashboard is not well defined, preventing objective testing
FR-08	Completeness, testability	“Throughout the lifecycle” does not identify the lifecycle states or the events that cause status changes. It could be unassigned, in progress, completed, or delayed. This reduces its completeness due to unclear understanding of status changes, making the required state changes unable to be objectively tested.
FR-09	Clarity, Consistency	The phrase “the case is released” is unclear because it does not define what “released” means. Does “released” mean that the case is no longer active for the unit, or does that the case is resolved? FR-11 instead states that the case is marked resolved/closed and the unit is released, so the

		two requirements also use inconsistent terminology for the same process.
FR-10	Clarity, Completeness, Correctness, Testability	“Notify the assigned case” does not identify a proper recipient. A case is not the intended notification recipient, making the stated behaviour incorrect, and there is no defined recipient for notification behaviour to be tested.
FR-11	Clarity	This requirement clarifies what FR-09 does not elaborate on: while FR-09 states that the unit’s availability updates when the case is released, it does not specify what action triggers the “release”. The term “resolved/closed” is unclear because it does not specify whether resolved and closed are two separate case statuses or two terms for the same status.
FR-12	Completeness, testability	The requirement clearly explains that the system should allow the dispatcher to search through the history logs but is less clear on how the history logs are searched, even though date, caller, and ID are stated. It could be clearer on how this information contributes to finding the correct call log. For example, whether you need all this info to search for a log or just enter any of this info to get a list of logs. The requirement is incomplete, which in turn makes it difficult to test.
NFR-Po1	Completeness, Correctness, Testability	The requirement does not define how the “most suitable department” is determined. It also refers to a single department, while the proposed system allows a case to be routed to multiple relevant departments. This routing is not fully defined, making the 2 second routing requirement difficult to objectively test.
NFR-Po2	Clarity, Completeness, Testability	“Performance Issues” is ambiguous. Without explicit details like acceptable response times or latency thresholds, it lacks clarity and completeness. This reduces testability, as performance testers have no criteria to determine the pass and fail conditions during load testing with 100 users.

NFR-Mo1	Clarity, Testability	The terms “module” and “entire system to crash” are not clearly defined. Some may interpret the boundaries of a module or system failure differently, making it difficult to determine objectively whether the requirement has been satisfied.
NFR-Mo2	<i>No significant issues</i>	No significant issue identified. The requirement clearly states the expected outcome that new user roles and departments can be added without modifying existing classes, which can be objectively checked when extending the system.
NFR-Ro1	<i>No significant issues</i>	No significant issue identified. The requirement clearly states the condition and expected outcome. It lets us know that when the data is being sent to the selected department, that data cannot be lost. This data being sent and received can be objectively tested fairly easily.
NFR-Ro2	Clarity, Completeness, Testability	This requirement specifies a 99.9% uptime, which is measurable, but it lacks clarity and completeness regarding boundaries. It doesn't define whether planned maintenance work counts towards downtime, or over what timeframe the 99.9% uptime is measured (week, month, year), or what critical parts of the system must be active to count the system as “running”. This uncertainty reduces testability.
NFR-Ro3	<i>No significant issues</i>	There is no significant issue with this requirement, it is similar to NFR-Ro1 where it is concerned about data being lost. This requirement clearly states the data that is being moved, and also the expected outcome that all submitted data must be saved without data loss. This can be objectively tested by comparing the stored data with the submitted data.
NFR-Ro4	Clarity, Completeness, Testability	In this requirement, the part “large number of cases” is ambiguous. Without a specific threshold, e.g. 1000 cases, the requirement is not complete or clear. This severely impacts testability, as the quality teams cannot establish a baseline for stress and load testing.
NFR-So1	<i>No significant issues</i>	No significant issue with this requirement. The requirement clearly states that any data being transferred between any user and the system must

		be encrypted. It can be objectively verified whether the transmitted data is encrypted.
NFR-So2	Completeness, Testability	This requirement establishes role-based access. However, testability and completeness rely on section 1.1. If this section definition of permissions is incomplete, it can introduce consistency gaps, preventing testers from validating the access boundaries.
NFR-Uo1	Completeness, Testability	The requirement mentions role-tailored views. However, it lacks completeness, missing the specific mappings per role. Without a table showcasing exactly what UI components belong to which user role, UX testers cannot test for correctness or verify whether unauthorised UI elements are correctly hidden.
NFR-Uo2	Clarity, Completeness, Testability	The term “plain English” in this requirement is subjective and can be interpreted differently for each person. This requirement lacks clarity and completeness because it doesn’t define the standards or error-handling guidelines. This lack of criteria makes it difficult to test this requirement.
NFR-Uo3	Testability, Completeness	This requirement states a “1-hour training limit”, which provides a clear metric, but it is incomplete on the evaluation parameters. It doesn’t define what the core functions of their role are or what the success rate counts as “complete”. Making objective testing hard

3.4 Improved and more testable requirements

- **FR-01** The system shall provide an interface for the dispatcher to record an emergency call, requiring the caller information, incident type, incident location, incident description, timestamp, and relevant dispatch units required to go on the scene.

- **FR-02** The system shall allow the dispatcher to create an emergency case from a recorded call, capturing all the details provided in FR-01.
- **FR-03** The system shall allow each emergency response department to view an interface and update the availability status, unit type, location, identifier, and count of the response units managed by that department.
- **FR-04** The system shall automatically determine the priority level of a newly created emergency case using the predefined decision process for the relevant emergency response department, based on the incident information recorded by the dispatcher. The priority determination processes for Medical, Police, and Fire cases are defined in Appendix 1.
- **FR-05** The system shall automatically route a case to the emergency response departments based on which dispatch units were assigned to the case.
- **FR-06** Upon a case being routed to a department, the department's system shall automatically assign the units from that specific department based on whether the unit is available. If multiple units are available, the system shall assign the first unit in the department's unit list. If no units are available, the case shall remain in a queue until a unit becomes available.
- **FR-07** The system shall provide each emergency response department with a dashboard displaying the current cases assigned to that department. Any changes to the case information should be updated on the dashboard without needing the user to manually refresh.
- **FR-08** The system shall update the status of each case by following these states: "Open" when a case is created, not yet assigned to a unit, "In Progress" when at least one response unit is assigned and has not signed off, and "Closed" when all response units assigned to the case have signed off. If all assigned units are removed before completing the response, the case shall return to "Open".
- **FR-09** When a unit is assigned to an active case, the system shall change the unit's availability status to "not available". When that response unit signs off from the case after completing its response, the system shall change its status to "available".

- **FR-10** The system shall notify the relevant unit by a notification displayed on the dashboard of the system. The notification should contain the assigned case with its case details upon assignment.
- **FR-11** The system shall allow each response unit assigned to a case to sign off when it has completed its response. The system shall automatically close the case once all response units assigned to that case have signed off.
- **FR-12** The system shall allow the dispatcher to search the history of logs using any combination of date, caller name, or case ID, returning all logs that match at least one entered field.

Improved Non-functional Requirements

- **NFR-P01:** When a completed case is submitted for routing, the system should send the case to all relevant emergency response departments within 2 seconds.
- **NFR-P02:** The system shall support at least 100 concurrent users while maintaining a response time of 2 seconds or less when navigating between interfaces or refreshing a view.
- **NFR-M01:** The system shall be built using a modular architecture such that the core dispatch functions (case creation, unit assignment, status updates) are not affected by non-critical modules (notification, search logs). **Check Appendix 2**
- **NFR-M02:** The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.
- **NFR-R01:** Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department.

- **NFR-R02:** The system should maintain at least 99.9% availability over a week; this includes the systems for Caller, Dispatcher, Field Responders, and Emergency Departments. Planned maintenance work does not count towards this downtime for any area.
- **NFR-R03:** All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered.
- **NFR-R04:** When 100 cases are open simultaneously by the dispatchers, response units and emergency departments, the system shall continue accepting, displaying, and updating cases without crashing.
- **NFR-S01:** Any data that passes between users and the system must be encrypted to prevent unauthorised access.
- **NFR-S02:** The system shall restrict each authenticated user to only the system features and information permitted for their assigned role. The role permissions are defined in Appendix 3.
- **NFR-U01:** The system shall display a tailored interface of the system for each authenticated user's assigned role, displaying only the tools and information required for that role. The roles and role permissions are shown in Appendix 3
- **NFR-U02:** The system shall return all error messages in plain language so they can be easily understood. It should also provide an action that can resolve the error and let the user know what to do. Technical error messages such as exceptions, stack traces, and debug information should never be displayed to the users.
- **NFR-U03:** After no more than 1 hour of training, each user shall be able to complete all defined core tasks for their assigned role without assistance. The core tasks for each role are listed in Appendix 4.

3.5 Defined acceptance criteria for the key features

These acceptance criteria are based on core functionality features in the emergency dispatch systems

Requirement	Acceptance Criteria
FR-01	● Given a dispatcher is on the call interface, when they attempt to submit the call record, they are required to fill in the caller information, incident type, incident location, incident description, and timestamp. ● Given that the dispatcher is recording a call, when they submit without entering all the details, then the system would reject the submission and inform the dispatcher to input all fields. ● Given a dispatcher is recording a call, when they attempt to submit the record without selecting a required dispatch unit type, then the system prevents the submission of the record.
FR-02	● Given a dispatcher has completed recording the call, when they create a case from that record, then a new emergency case shall be created containing all the recorded details defined in FR-01. ● Given a case is newly created from a recorded call, when creation completes, then the case status is set to “Open” with no units assigned to it yet.
FR-04	● Given a dispatcher has submitted a case, when the system processes the case, then the system should automatically assign a priority level without manual dispatcher input. ● Given a case has been classified under Medical, Police, or Fire, when the system determines priority, then it should apply the decision process defined for that department (Appendix 1). ● Given the information recorded by the dispatcher, when the system runs it through the decision process, then the resulting priority level should match the outcome defined by the process.

FR-05	● Given that the dispatcher has created a case, when the case contains a specific unit type (police, fire, ambulance), then the case is automatically sent to the corresponding department (police to police department, fire to firefighter department, and ambulance to the Medical department). ● Given that the dispatcher has created a case with more than one dispatch unit type, when the case is ready for routing, then the case would automatically be sent to the corresponding departments.
FR-06	● Given a case has been routed to a department, when the department's system receives the case, then it shall automatically assign a unit from that department. ● Given a department has both available and unavailable units, when the system is assigning a unit, it should only assign units that are labelled "available". ● Given all units in a department are unavailable, when a case is routed to that department, then the case shall remain in the queue until a unit becomes available.
FR-07	● Given cases have been routed to a department, when a user opens that department's dashboard, then it shall display all cases that are assigned to that department, and no other cases. ● Given a department's dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically. ● Given a department's dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically.
FR-08	● Given a new case is created, when it has not been assigned to any unit, then the status of that case should be "open". ● Given a case has status "open", when it has been assigned to a unit, then the status should change to "in progress". ● Given a case has status "in progress", when some units have signed off but at least one assigned unit is still active, then the case shall remain "in progress" ● Given a case has status "in progress", when all the units assigned to that case have signed off, then the system shall update its status to "closed". ● Given a case has status "in progress", when all assigned units are removed before completing the response, then the case status should return to "open".

FR-09	● Given a response unit is labelled “available”, when the unit is assigned to an active case, then the system shall change its availability status to “not available”. ● Given a unit is currently assigned to a case, when the unit signs off, then the unit status would change from “not available” to “available”.
FR-11	● Given a response unit is assigned to a case, when the unit has completed its response, then the system shall allow the unit to sign off from that case. ● Given a unit is currently assigned to a case with other units, when one unit signs off while at least one other unit remains active, then the case shall remain “in progress” ● Given multiple units are currently assigned to a case, when the final remaining active unit signs off, then the case shall switch to “closed”

3.6 Identify relevant software quality attributes *(maybe delete this section)*

The main software quality attributes relevant to the proposed system are reliability, usability, maintainability, performance, and security. Reliability, usability and maintainability relate directly to the main quality issues that were mentioned in Section 1.2.

The other quality issues; performance and security, are also important because the system will deal with users sensitive information and is time sensitive. These quality attributes are addressed in Section 3.2 and Section 3.4.

4. Initial Prototype

- ☒ Develop an initial prototype that demonstrates the main intended functionality.
- ☒ Include a GUI if the solution involves user interaction, data entry, workflow processing, search, reporting, dashboards, or end-user decision support.

5. AI-Assisted Development Using GitHub Copilot

- ☐ Use GitHub Copilot to support prototype development and QA activities.
- ☐ Identify which parts of the prototype or QA artefacts were supported by AI suggestions.
- ☐ Provide examples of prompts, comments, code suggestions, generated snippets, or AI-assisted outputs.
- ☐ Identify any limitations, errors, risks, bias, security issues, maintainability issues, or quality concerns found in AI-assisted code.
- ☐ Reflect on how AI-assisted coding supported learning, productivity, testing, documentation, or quality awareness.

6. Initial Test Strategy and Test Planning

- ☐ Define the scope of testing and out-of-scope items.
- ☐ Identify relevant test levels or test types, such as unit testing, integration testing, system testing, acceptance testing, usability testing, regression testing, or security testing.
- ☐ Explain the functional testing approach.
- ☐ Explain the non-functional testing approach where relevant.
- ☐ Define entry criteria and exit criteria.
- ☐ Identify the test environment, tools, browsers, devices, platforms, or dependencies.

6.1 Scope of Testing

- ☐ In-scope: list your key features (same ones you wrote AC for) – case creation/recording, priority determination, routing, unit assignment (incl. double-dispatch), case status lifecycle, live dashboard
- ☐ Out-of-scope: things not yet testable at prototype stage – full security implementation, full-scale load testing (100+ users), real-user usability testing, notification/log search modules (deprioritised features)

The scope of testing defines what areas of the Emergency Dispatch Priority & Coordination System provide test evidence during the testing phase. The testing will focus on the core functionality of the system, and quality risks that could result in incorrect information, incorrect priority management, incorrect dispatching of units, delayed response on scene, or incorrect states for units and cases.

Exhaustive testing is not logical because it is impractical to test every possible input, sequence and condition. So instead, a **risk-based testing** approach will be used. This means that features where failure could have the biggest impact will get more in-depth testing. Attention will be given to any complicated decision logic, interactions between the system components, and places where system defects can lead to a response unit or emergency case with the incorrect state.

Test Item	Requirement	Scope	Risk
Emergency call recording	FR-01	Recording required information about the caller, incident, location, emergency service types, input validation, missing information, and invalid values.	High
Case creation	FR-02	Creating a case from a completed call and verifying that all recorded information is persisted; the case receives the correct initial status, and incomplete or duplicate actions are handled correctly.	Critical
Priority determination	FR-04	Applying the priority rules for Police, Fire, and Ambulance to incident information and verifying the priority outcome, including relevant boundary conditions.	Critical
Department routing	FR-05	Routing cases to the correct department according to the emergency service types required by the case, including cases that require multiple departments.	Critical

Unit assignment	FR-06	Assigning only suitable units that belong to the correct department based on their availability and distance to the incident. Cases with no available units remaining must wait for assignment.	Critical
Concurrent unit assignment	FR-06 / NFR-R04	Ensuring simultaneous assignments cannot allocate the same response unit to more than one case at once.	Critical
Department dashboard	FR-07	Displaying the correct department's active cases and updating case information without needing to manually refresh.	High
Case status lifecycle	FR-08	Correct transitions between Open, In Progress, and Closed, including cases where there are multiple units assigned, and cases where assigned units are removed. Invalid state transitions will also be tested.	Critical
Unit availability lifecycle	FR-09	Correct transitions between Available and Unavailable since units are assigned to and released from cases.	Critical
Response completion and sign-off	FR-11	Individual unit sign-off and automatic closure of a case, which occurs only after all units have completed their response.	High

Testing these functions is important because individual components could work correctly in isolation, but defects can still occur when there is communication between multiple components. For example, case creation and priority determination could both work perfectly in isolation. However, in an integration test where both components are communicating, an integration defect could cause the case information not to be passed properly to the priority determination component.

Non-functional quality concerns will also be included in testing where it is possible to evaluate them in the current environment. These areas are included in the testing scope because they directly relate to the major quality risks that were identified earlier in the report.

- **Reliability** can be evaluated by testing the consistency of case data during case creation, routing, assignment, and status changes.
- **Concurrency** will be tested by trying to assign the same response unit to multiple cases simultaneously.
- **Performance** can be tested by measuring the time it takes for critical operations to complete. This includes department routing and auto-refreshing of the interface.
- **Security** and authorisation can be tested using simple verification and by making sure system roles cannot access functions outside their permissions.
- **Usability** will be evaluated through basic checks of the interface, making sure error messages are clear, and whether users can perform the core tasks without asking for help.

Some areas will not receive validation during the initial test cycle. These features include:

- Large-scale testing with 100 or more simultaneous users
- Long-term verification on the 99.9% uptime requirement
- Full security testing
- Recovering after system outages
- Integration with real NZ departments
- Large-scale usability testing involving real-life stakeholders
- Overall features that are not yet implemented in the prototype, like historical log searching or advanced notification functionality.

Because there are lots of core functions to test, the requirements with the most risk exposure will have the highest priority. This includes priority determination, department routing, response unit assignment, case and unit state consistency, and persistence of emergency case information.

These areas present the greatest risk because any defects in these areas could result in an emergency being incorrectly prioritised, sent to the wrong service or assigned to an unavailable unit.

6.2 Test Types / Levels

- ☐ Short description of each level + why it's relevant to your project:
- ☒ ~~Unit testing — individual logic (priority decision, unit assignment)~~
- ☒ ~~Integration testing — module handoffs (case creation → routing → assignment)~~
- ☒ ~~System testing — full end-to-end flow~~
- ☒ ~~Acceptance testing — running your AC scenarios directly~~
- ☒ ~~Regression testing — re-verifying fixes, especially around the race condition~~

Test levels and types are relevant and would be beneficial to implement during the testing phase on the Emergency Dispatch system. Below are testing levels and types that should be implemented:

Unit Test

Unit testing plays a crucial role in the emergency dispatch system, as it is great for testing individual sections of the code, for example, a single class or a method in a class. This is an incredibly easy way to test boundaries and logic of the code. In the emergency dispatch system, testing plays an important role in testing the priority determination logic and the assignment of case logic in the code. For example, a unit test could be used to test the boundaries set for the record details fulfillment when the dispatcher inserts the caller's details information (testing the input and input type). Another example of another method requiring a unit test is the priority determination, as each priority is determined by a standardised activity diagram(Appendix 1). Therefore, unit testing would be good to test all possible predefined outcomes of each incident information.

Integration Test

For integration testing, it is used to test different services or different modules in the system. In the emergency dispatch system, there are multiple services, including assigning a unit to a case. Assigning the unit to a case requires both a unit and a case status to be correctly synchronised and requires changing the domain objects of the unit and case. For example, the unit status would need to change to

“unavailable,” and the case status would need to change to “In Progress” when a unit is selected to solve the case. Without integration testing, we would not be able to test the synchronisation of both unit and case status, which is an essential part of the emergency dispatch system functionality.

System Test

System testing is used for testing the relevant non-functional and functional requirements. This testing is required for multiple workflows that exist in the emergency dispatch system. An example that system testing could be used is recording a case from a caller to resolving the case. This whole process is tested with system testing, which covers multiple functional and non-functional requirements in order to test how a normal use of the system would function. A fault could occur in the process where the individual testing (integration and unit) may not necessarily be able to identify the cause, as they test individual components rather than the whole process.

Acceptance Test

Acceptance testing is valuable in a system like the emergency dispatch system, as it's built for the stakeholders (dispatcher, emergency departments, units) to use; the system must be tailored to what they require. It has to support their operations, meet their expectations of how the system should operate, be fully operational, and satisfy their requirements. Acceptance testing is relevant to this system as it is needed to verify the requirements given by the stakeholders rather than just making sure the system developed is functionally working.

Regression Test

The modification of a section of code should not cause unwanted changes to other sections of the codebase; therefore, regression testing is used to ensure that code that is already fully functional and meets all the requirements could continue to behave normally even when new commits are made. Regression testing is a necessary testing type not just for this emergency dispatching system but also in general for most other systems. In particular for this system, this would be used to ensure that when synchronisation is considered and implemented, the original functionality of the unit assignment (no concurrent) would not be altered.

6.3 Functional testing approach

- ☐ How you'll test FRs: deriving test cases from acceptance criteria (already built in 3.5)
- ☐ Manual vs automated – state your team's current approach (manual first, moving to automated unit tests)
- ☐ Reference your testing framework once decided (MSTest/NUnit/xUnit)

To test the functionality of the emergency dispatch system, we will determine how the system operates by checking whether the behavior of the system matches the functional requirements (section 3.4) and the acceptance criteria defined for the key features (section 3.5).

The black box techniques would be utilized as the main approach. This approach focuses on testing the behavior of the system, how it will react to inputs and the output produced, without understanding its implementation. The following black box techniques would be used in our testing.

Equivalence partitioning could be used to test requirements. This is done by separating the input into groups that are expected to behave similarly. There will be a representative value from each group that will be tested, rather than testing every possible value. For example, in FR-04, the partitions for this requirement would be split into valid values (high, medium, low) and invalid values (undefined priority or no priority at all).

Next, boundary value analysis is used; this technique focuses on testing the edge cases because defects often occur near the edges of valid and invalid ranges. For example in FR-06 when determining if a case needs to be in a queue or not depending on the available units, the boundaries include 0 units (case is queued), 1 unit available (case is assigned to unit) and 2 units available (first unit on the list gets assigned to the case).

A decision table could be also used as it would be helpful in determining the outcome of different combinations of outputs. In our system this would be used on FR-08, where a case with multiple unit types did not sign off the case then the status would still display "In Progress", it would only become "closed" if all unit types assigned are all signed off.

The last technique is state transition testing, where it focuses on checking that the state changes are correct. This could also be used for testing FR-08.

These testing would be completed by a combination of automated testing and manual testing depending on the requirements as well as to double check whether the system is working as intended

6.4 Non-Functional Testing Approach

- ☐ Map specific NFRs to how you'll test them:
- ☐ NFR-PO1/PO2 (performance) → timing/benchmark tests
- ☐ NFR-RO4 (concurrent cases) → concurrency/race condition testing (ties directly to FR-O6)
- ☐ NFR-MO1/MO2 (maintainability) → design/code review against Appendix 2's module table, rather than automated tests
- ☐ NFR-SO1/SO2 (security) → likely out-of-scope for now, or basic checks only

Because our main non-functional requirements span over 4 categories: performance, reliability, maintainability, and security, we need to use more than one testing method. Each of these categories need their own approach because testing security will be different from testing performance or maintainability.

Using timing and benchmark tests is the approach for testing performance. For NFR-PO1, the time passed between the case being submitted and received from the directed department will be measured. To confirm this requirement, the time measured will have to be under 2 seconds. For NRF-PO2, smaller tests will be run, which include simulating around 10 case submissions at once. The goal of this test is to see the response times when navigating or refreshing the page, to see if the time taken to refresh is logical. This gives us an indication of what the performance will be like as we scale the system. Smaller tests will be applied because testing 100 concurrent users is out of scope for the prototype (noted in 6.1).

The main focus of reliability tests is to test concurrency/race conditions. For NFR-RO1, there will need to be verification that the data was saved successfully, and that what was entered by the dispatcher matches the output on the receiving department's interface. NFR-RO4 focuses on concurrency and race-condition testing. Our 6th functional requirement states that the system should assign units automatically. However, the problem statement identified the risk of a unit being dispatched twice at

the same time. To test this, we will simulate multiple concurrent case assignments to the same unit and make sure the system only allows the unit to be assigned to one case at a time.

6.5 Entry and Exit Criteria

- ☐ Entry criteria: e.g., code compiles, feature complete, AC defined, test environment ready
 - ☐ Exit criteria: e.g., all key AC pass, no critical defects open, race-condition test passes under concurrent load
- Creating in local environment, adding to doc on Saturday - fateh

6.6 Test environment, tools, and dependencies

- ☐ Language/framework: C#, .NET, Visual Studio
- ☐ Testing framework: (MSTest/NUnit/xUnit – still need this from you)
- ☐ Version control: GitHub (Leo, Fateh, Shawn's shared repo)
- ☐ Any other dependencies (mock data, in-memory repository pattern, etc.)
- ☐ Platform: likely Windows/desktop console or GUI environment – confirm

Test Environment

For the test environment, it would be tested on the local machine,

The emergency dispatch system would be developed on Visual Studio. The system would be developed using C# on the backend and HTML on the frontend since this is a web application, so that callers, dispatchers, and the emergency department can easily access this system.

Testing

7. Initial Test Cases and Requirements Traceability

- ☐ Design at least eight (8) initial test cases.
- ☐ Ensure the test cases cover key functional requirements and selected non-functional requirements where appropriate.
- ☐ Create an initial requirements traceability matrix linking requirements to test cases.
- ☐ Explain how traceability supports quality assurance and change management in the project.

- Creating in local environment, adding to doc on Saturday - fateh

8. Project Progress, Risks, and Next Steps

- ☐ Summarise work completed so far.
- ☐ Identify current limitations, risks, and challenges.
- ☐ Explain how risks will be managed in the next phase.
- ☐ Provide a clear plan for completing the final project.
- ☐ Include a brief team reflection on AI-assisted learning, coding, testing, quality assurance, and group collaboration.

Appendix

Appendix 1. Priority/severity activity diagrams for Medical, Police, Fire

Appendix 2. Critical and Non-Critical modules

Appendix 3. Roles and role permissions

Appendix 4. Core tasks for each user role